Bank Licensing & Compliance Portal

High-Level Architecture Document

Document property	Details
Document Title	Bank Licensing & Compliance Portal Architecture Design
Version	1.1
Status	2nd Written Test
Technology Stack	Node.js · PostgreSQL · React.js
Date	11 May 2026

TABLE OF CONTENTS

1. Introduction 1
1.1 Purpose 1
1.2 Background & Problem Statement 1
1.3 Scope 1
1.4 Technology Stack 1
2. Functional Requirements 1
2.1 User Registration & Onboarding 1
2.2 Authentication & Session Management 1
2.3 Role-Based Access Control 1
2.4 Application Types & Document Requirements 1
2.5 Application Lifecycle & State Machine 1
2.6 Stage Decisions 1
2.7 Document Handling 1
2.8 Audit Trail 1
2.9 Concurrent Access Handling 1
3. Non-Functional Requirements 1
4. System Architecture 1
4.1 Component Overview 1
4.2 API Layer Structure 1
4.3 Authentication Flow 1
4.4 State Transition & Stage Decision Flow 1

5. Data Flow 1
5.1 Applicant Self-Registration Flow 1
5.2 Staff Account Creation Flow 1
5.3 Application Submission Flow 1
5.4 Review, Stage Decision & Approval Flow 1
5.5 Document Version Flow 1
6. Database Design 1
6.1 Schema Overview 1
6.2 Table Definitions & Constraints 1
6.2.1 users 1
6.2.2 institutions 1
6.2.3 user_workflow_roles 1
6.2.4 application_types 1
6.2.5 workflows 1
6.2.6 workflow_states 1
6.2.7 workflow_transitions 1
6.2.8 document_requirements 1
6.2.9 applications 1
6.2.10 application_documents 1
6.2.11 application_stage_reviews 1
6.2.12 audit_logs 1
7. Relationships & Logical Data Model 1
7.1 Entity Relationships 1
7.2 Logical Data Model, Key Invariants 1

7.3 Indexes 1
8. Data Specifications 1
8.1 API Response Envelope 1
8.2 HTTP Status Code Specification 1
8.3 Core Data Types & Validation Rules 1
8.4 Application Timeline Response 1
8.5 Seed Data Specification 1
9. Security Design Decisions 1
9.1 JWT vs Session Based Authentication 1
9.2 Two Tier Role System 1
9.3 Reviewer Cannot Be Approver 1
9.4 Append Only Audit Log & Stage Reviews 1
9.5 Concurrent Access 1
10. Testing Strategy 1
11. Assumptions & Open Decisions 1
12. Future Enhancements 1

1. Introduction

1.1 Purpose

The document presents the architecture of the Bank Licensing & Compliance Portal developed for the National Bank of Rwanda (BNR). This includes details on functional and non-functional requirements, system architecture, data flow diagrams, database schema, and logical data modeling. The target readers of this document are the engineering team and the technical acceptance authority at NBR.

1.2 Background & Problem Statement

The current licensing process in BNR is conducted manually with applications being submitted via e,mail, spreadsheet tracking of documentation, and manual approval. The result is that there is:

• No central system of tracking application status

• No clear audit trail appropriate for any legal and regulatory requirement

• Little visibility for the applicant about the status of its application

• High probability of mistakes, late submissions, and non-uniform decision-making

The portal provides a structured and auditable digital system for managing applicant self-registration, staff onboarding, multistage regulatory review with stage,wise decisions, and recording of approvals.

1.3 Scope

It encompasses the entire process for a banking license application, which includes registration of the applying financial institution, administration,led onboarding with workflow role assignments, document uploading, multistep review process and stage decisions, and finally approval/rejection of the application. This particular system excludes payment handling, email notifications, and integration with any third party data sources from its scope.

1.4 Technology Stack

Layer	Technology	Justification
Backend API	Node.js 20 LTS + Express.js	Asynchronous I/O fits well with concurrent document uploads, extensive ecosystem, and familiarity with the team.
Database	PostgreSQL 16	ACID transactions, row, level locking, JSONB fields, granular permissioning, everything needed for auditing and concurrency.
Frontend	React.js 18 + React Router	Component architecture aligns perfectly with role, based view processing; easy to understand for internal applications.
Authentication	JSON Web Tokens (JWT)	Stateless nature, horizontal scalability, great fit for API, first design; access tokens with refresh token rotation.

Layer	Technology	Justification
ORM / Query	Knex.js (query builder)	Very lightweight; allows explicit transaction and lock management without obscuring SQL
File Storage	Local filesystem (simulated)	Derived from requirements; stored in PostgreSQL database; abstraction provided for later cloud transition

Testing	Jest + Supertest	Tests including unit and integration testing; Supertest library for HTTP authentication checks

2. Functional Requirements

2.1 User Registration & Onboarding

The system supports two distinct registration paths depending on who the user is:

User Type	Registration Method	Who Controls It	Active Immediately?
APPLICANT	Self-registration via public endpoint	The applicant themselves	Yes
INTAKE_OFFICER / LEGAL_OFFICER / FINANCIAL_OFFICER / REVIEWER	Created by ADMIN via protected endpoint	BNR Administrator	Yes, temporary password
APPROVER	Created by ADMIN via protected endpoint	BNR Administrator	Yes, temporary password
ADMIN	Seed scripts only never via API	System / DevOps	Yes, seeded at deployment

The applicant registration procedure entails the collection of individual account data together with institutional data in one atomic process. Staff members' accounts are created through the admin, who is also responsible for specifying the workflow role associated with the staff member during the account creation process.

2.2 Authentication & Session Management

• User registers and logs in using email and password (bcrypt hashing)

• Access token generated upon login, 15 minutes expiration; includes only user ID and system role in token payload

• Refresh token saved to http Only cookie 7 days expiration with renewal after each usage

• Server checks token signature and expiration for all secured routes

• Fine-grained workflow role of staff members is not included in the JWT token; role obtained freshly from user workflow roles upon each transaction

• Must change password parameter is returned as 'true' in response upon first login attempt for Admin created users

2.3 Role, Based Access Control

Two different levels of access are defined by the system. The first is based on system roles (course grained, implemented in JWT), while the second is based on workflow roles (fine grained, implemented in the database).

System Role	Description	Key Permissions
APPLICANT	Representative of a financial institution applying for a license	Self-rgister; submit own applications; upload documents; view own application timeline;

STAFF	BNR employee, covers all internal review roles	Access staff dashboard; act on applications within their assigned workflow role; post stage decisions
ADMIN	System administrator	Create and manage user accounts; assign workflow roles; read-only access to all applications and audit logs

Workflow Role	**Assigned To**	**What They Can Do**
INTAKE_OFFICER	STAFF users	Retrieve applications; do preliminary assessment; request additional information; pass to next level
LEGAL_OFFICER	STAFF users	Do legal assessment; make decision at this stage; pass on to financial assessment
FINANCIAL_OFFICER	STAFF users	Do financial assessment; make decision at this stage; pass on to approval stage
REVIEWER	STAFF users	General reviewer function is used in more straightforward processes (e.g. Microfinance); covers both intake and assessment activities
APPROVER	STAFF users	Make final decision (approval or denial); mandatory: must not be the same individual who assessed the application

2.4 Application Types & Document Requirements

• The required/optional documents are defined by the application type (Commercial Bank License, Microfinance Institution License etc.)

• The document requirement list resides in the database and dynamically retrieved, no hardcoded types in application code

• Server-side validation will block submission if the required documents according to that application type are not provided in the present submission round

• Addition of a new license type requires merely creation of seed/migration scripts, no change in code needed

2.5 Application Lifecycle & State Machine

Each application follows a well-defined workflow according to the type of application. The state machine is implemented at the API level any invalid transitions will be denied using HTTP 422. The terminal states (APPROVED and REJECTED) do not have any outgoing transitions and are immutable.

From State	**To State**	**Role Required**	**Notes**
DRAFT	SUBMITTED	APPLICANT	Blocked if required documents are missing
SUBMITTED	UNDER_INITIAL_REVIEW	INTAKE_OFFICER	Sets reviewed_by = acting user ID
UNDER_INITIAL_REVIEW	PENDING_INFORMATION	INTAKE_OFFICER	Stage decision REQUIRED — must include written decision rationale
PENDING_INFORMATION	SUBMITTED	APPLICANT	Applicant re uploads documents; submission cycle increments

UNDER_INITIAL_REVIEW	UNDER_LEGAL_REVIEW	INTAKE_OFFICER	Stage decision REQUIRED
UNDER_LEGAL_REVIEW	UNDER_FINANCIAL_REVIEW	LEGAL_OFFICER	Stage decision REQUIRED
UNDER_FINANCIAL_REVIEW	PENDING_APPROVAL	FINANCIAL_OFFICER	Stage decision REQUIRED
PENDING_APPROVAL	APPROVED	APPROVER	Must not be same user as reviewed_by; stage decision REQUIRED
PENDING_APPROVAL	REJECTED	APPROVER	Must not be same user as reviewed_by; stage decision REQUIRED

For any transition of stages that moves the application further away from being “SUBMITTED”, a stage decision document will be needed, and if there is no such document, the request will not be honored.

2.6 Stage Decisions

A decision regarding the stage will be registered whenever a staff member either promotes or reverts an application. It is done in one single transactional manner, whereby the decision and the stage change are committed or rolled back together.

• Decision category: APPROVED STAGE, REQUEST INFO, or ESCALATED

• A decision explanation is required in writing; the API checks whether decision note is non-empty: CHECK (length (trim (decision note)) > 0)

• Decision logs are append only; once created, they cannot be edited or deleted.

• Decision logs are viewable by anyone with permission to view the application, including the applicant themselves – transparency is one of the fundamental principles behind the system design.

• The applicant can access the complete history of stage decisions in his application profile page, where he receives a written explanation of each move.

2.7 Document Handling

• Applicant uploads documents corresponding to certain requirement keys based on the type of application

• Server imposes a 5 MB limit on the size of file that it accepts

• The content of the file is saved in the local file system, while its metadata is saved in the database

• If an applicant re-submits following the PENDING INFORMATION status, the submission cycle increases, but all other documents remain available without being deleted

• The history of documents uploaded can be seen for all submission cycles by the reviewers using the history endpoint

2.8 Audit Trail

• All state changes, stage decisions, file uploads, logins, and other administration tasks are logged to the audit logs table

• The logging is done in an append only fashion there are no UPDATE or DELETE statements for this table in the application's source code; the application user on PostgreSQL has permission to INSERT records only

• Each record contains who, what, app id, pre-state, post-state, time stamp, and metadata in a JSONB format

• The logging is meant to be presented as evidence in court just immutable facts

2.9 Concurrent Access Handling

• Application uses version number column to support optimistic locking

• Each state change will be carried out through a SQL update statement: "UPDATE applications SET state=? version=version+1 WHERE id=? AND version=?"

• In case there are no affected rows, the state change fails with HTTP code 409 Conflict, indicating another user beat you to the punch

• There is only one database transaction involved, either both succeed or both fail

2. Non-Functional Requirements

Category	Requirement	Target / Approach
Security	Authentication	JWT 15,min expiry; refresh tokens in httpOnly cookies; bcrypt cost factor 12
Security	Authorization	Backend middleware validates system role on every request; workflow role resolved from DB on every action; frontend hides inaccessible actions but backend always re-checks
Security	Audit integrity	Append only audit table; DB user has INSERT only; no update or delete method exists in application code
Security	Input validation	All inputs validated with Joi before hitting service layer; parameterized queries only no raw SQL interpolation
Reliability	Concurrent safety	Optimistic locking on applications table; 409 returned on version conflict
Reliability	Transaction integrity	Stage decision write and state transition occur in one transaction — atomic commit or rollback
Reliability	Data permanence	Terminal states have no outgoing transitions; stage decision records and audit logs are immutable
Scalability	Stateless API	JWT tokens are stateless; API tier can scale horizontally without session affinity

Scalability	Database	PostgreSQL with indexes on FK columns, status columns, and created at for query performance
Maintainability	Workflow as data	Workflows, states, and transitions are database records; new license types require no code change
Maintainability	Error handling	Global Express error handler; no stack traces in API responses; structured JSON error objects always
Compliance	HTTP status codes	401 unauthorized, 403 forbidden, 409 conflict, 422 processable consistently applied
Usability	UI role fidelity	Actions the user cannot perform are hidden from the UI; loading, error, and empty states explicitly handled
Transparency	Stage decision visibility	All stage decisions visible to applicant. no hidden internal tier; promotes regulatory transparency

4. System Architecture

4.1 Component Overview

The software architecture utilizes a 3-layer architecture consisting of the presentation layer with a React.js SPA, the application layer with Node.js / Express REST API, and the data layer with PostgreSQL DB. Local file system is used for file storage, while metadata about those files is stored within the database.

Component	Technology	Responsibility
Web Client (SPA)	React.js 18	Renders UI based on authenticated user's system role and workflow roles; calls REST API; handles loading, error, empty states
API Server	Node.js + Express	Validates JWT, enforces RBAC, executes business logic, manages state transitions, writes stage decisions and audit logs
Auth Middleware	express, JWT + custom	Decodes and verifies JWT on every protected route; injects user context into request object
RBAC Middleware	Custom Express	Checks system role against route permission map; returns 403 if denied
State Machine Service	Pure Node.js service	Reads workflow transitions from DB; validates transition legality; enforces reviewer, cannot, approve rule; requires stage decision on advances
Stage Decision Service	Pure Node.js service	Validates decision object; inserts into application stage reviews within the same transaction as the state transition

Document Service	Multer + Node.js	Handles multipart uploads; enforces 5 MB limit server-side; writes to file system; records metadata in DB
Audit Service	Node.js service	Called within every DB transaction that modifies application state; inserts append, only audit record
Database	PostgreSQL 16	Stores all persistent data; enforces FK constraints and CHECK constraints; controls audit table privileges
File Storage	Local filesystem	Stores binary file content; path recorded in application documents; abstracted behind Document Service

4.2 API Layer Structure

Express App is organized in layered modules:

```
src/
  routes/          HTTP route definitions, input validation (Joi)
  middleware/      auth.js, rbac.js, errorHandler.js, upload.js
  services/        stateMachine.js, stageDecision.js, auditService.js, documentService.js
  controller/      applicationController.js, userController.js, auditController.js, documentController.js, WorkflowController.js
  repositories/    applicationRepo.js, userRepo.js, auditRepo.js, documentRepo.js, stageDecisionRepo.js
  dto/             handle request validation
  db/              knex instance, migrations/, seeds/
  utils/           jwt.js, password.js, errors.js
  app.js           Express setup
  server.js        HTTP server entry point
```

4.3 Authentication Flow

1. Client makes POST request to /api/auth/login endpoint with username and password

2. Bcrypt checks password against bcrypt hash from the users table

3. If must change password is equal to true: response 200 OK and { password change required: true } not yet authorized fully

4. After successful authentication: create access token using JWT (expires in 15 min, has user id and role); store refresh token in httpOnly cookie

5. Client stores access token in memory and uses Authorization: Bearer <access token> header for all future requests

6. auth middleware validates JWT on each protected route

7. rbac middleware compares role to permissions of the current route; forbidden response (403 status code) if permission denied

8. If operation requires workflow role: state machine retrieves user's workflow roles from database and checks against role of transition

4.4 State Transition & Stage Decision Flow

Every state change for an application that is progressing requires the creation of a stage decision:

9. Post /api/applications/:id/stag-,decision body { toState, decisionType, decisionNote }

10. Authentication middleware authenticates JWT; rbac middleware checks STAFF system_role

11. State machine service finds the application and workflow

12. State machine service finds if the acting user has the required workflow role for the transition

13. State machine service verifies: (a) transition exists in workflow, (b) application not in terminal state, (c) if an approval: reviewed_by !== acting user

14. Stage decision service verifies: decisionNote is a non-empty string

15. Checks passed – transaction begins

16. UPDATE applications SET current_state=?, version=version+1 WHERE id=? AND version=? – Optimistic locking check

17. If no rows were updated: transaction aborts, 409 Conflict returned

18. INSERT application_stage_reviews in the same transaction

19. INSERT audit_logs in the same transaction

20. Transaction commits; API returns updated application and created stage decision

5. Data Flow

5.1 Applicant Self-Registration Flow

Data flow from registration to first application submission:

21. Applicant makes a POST /api/auth/register call with email, password, full_name, phone, institution_name, and institution_registration_number

22. All parameters are validated; unique email and registration number are checked

23. Database transaction starts: INSERT INTO users where system_role = 'APPLICANT'; INSERT INTO institutions associated with the new user's ID

24. USER_REGISTERED audit trail is logged during the transaction

25. The transaction ends; an access JWT token and a refresh token are generated right away

26. Applicant logs in and lands on their dashboard; no email verification required in v1

5.2 Staff Account Creation Flow

Data flow when Admin creates a staff account with workflow role assignment:

27. Admin invokes POST /api/admin/users with fields: email, full_name, system_role: STAFF, workflow_roles

28. The API validates that the user making the request is of system_role: ADMIN

29. The server creates a random 16,character temporary password which gets bcrypt,hashed

30. The database executes an INSERT transaction: users(system_role: STAFF, must_change_password: TRUE)

31. For each item in workflow_roles: user_workflow_roles(user_id, workflow_id, role, assigned_by)

32. The audit log logs a USER_CREATED event made by the Admin

33. The transaction completes; the temporary password gets returned in the response which Admin can relay to the staff member

34. The next time the staff member logs in: The API sees must_change_password: TRUE and forces password change

5.3 Application Submission Flow

Path from application creation to application being successfully submitted:

35. Select application type via GET /api/application,types; response includes list of document requirements

36. Upload form dynamically created by frontend using document_requirements associated with that application type

37. Document uploaded through POST /api/applications/:id/documents/:requirementKey; Document Service checks validity of MIME type, size of document and saves it in application_documents

38. Clicking Submit sends POST /api/applications/:id/stage,decision { toState: SUBMITTED } to server

39. State machine checks that all required documents have been uploaded in the current cycle of submission before allowing transition

40. Application state becomes SUBMITTED; audit log recorded; applicant receives new status on dashboard

5.4 Review, Stage Decision & Approval Flow

Path for data within multiple stages of regulatory review process:

41. Intake Officer views all SUBMITTED applications on their dashboard filtered by workflow role using user_workflow_roles table

42. Intake Officer changes application status to UNDER_INITIAL_REVIEW user ID is saved under applications.reviewed_by

43. Intake Officer reviews the application and makes POST /api/applications/:id/stage,decision { toState, decisionType, decisionNote }

44. Stage decision is saved into database; application state changes atomically; an audit log entry is generated

45. Applicant immediately views the decision on their timeline; decision note is visible to applicant

46. When decision is REQUEST_INFO: application moves to state PENDING_INFORMATION; new documents are uploaded by applicant; submission_cycle increases; applicant submits again

47. Process continues from legal review to financial review and then back — at every stage stage decision is mandatory

48. Approver views application which reached state PENDING_APPROVAL API validates that reviewed_by != approver's user ID before proceeding

49. Approver makes final decision: APPROVED or REJECTED with decision note; stage is terminal; decision is logged permanently

5.5 Document Version Flow

When the applicant re,submits documents after a request for information:

50. The application goes into PENDING_INFORMATION with stage decision note specifying requirements.

51. The applicant submits the updated document using POST /api/applications/:id/documents/:requirement_key API.

52. The Document Service will generate a new application_documents row with submission_cycle being set as current_cycle +1; the existing application_documents.superseded_by should now point to the new row.

53. The applicant resubmits; the state machine checks the documents for the current submission cycle only.

54. The reviewer can check history using GET /api/applications/:id/documents/:requirement_key/history.

6. Database Design

6.1 Schema Overview

The database is structured into five functional groups, namely Identity and Access (users, institutions, user_workflow_roles), Workflow Configuration (application_types, workflows, workflow_states, workflow_transitions, document_requirements), Application Data (applications, application_documents), Stage Decisions (application_stage_reviews), and Audit Trail (audit_logs).

6.2 Table Definitions & Constraints

6.2.1 users

Column	Type	Constraints	Description
id	UUID	PK DEFAULT gen_random_uuid()	Surrogate primary key
email	TEXT	NOT NULL, UNIQUE	Login identifier, lowercase, normalized before storage
password_hash	TEXT	NOT NULL	bcrypt hash of user password
full_name	TEXT	NOT NULL	Display name used across all UI surfaces
phone	TEXT		Contact number, optional for staff, captured for applicants

system_role	TEXT	NOT NULL, CHECK IN ('APPLICANT','STAFF','ADMIN')	Coarse, grained system role controlling section access
must_change_password	BOOLEAN	NOT NULL DEFAULT FALSE	TRUE for Admin, created accounts; cleared on first password change
is_active	BOOLEAN	NOT NULL DEFAULT TRUE	Soft disable accounts without deletion
last_login_at	TIMESTAMPTZ		Updated on every successful login
created_at	TIMESTAMPTZ	NOT NULL DEFAULT now()	
updated_at	TIMESTAMPTZ	NOT NULL DEFAULT now()	

6.2.2 institutions

Captures the financial institution represented by each applicant. Created atomically with the user record during self-registration.

Column	**Type**	**Constraints**	**Description**
id	UUID	PK DEFAULT gen_random_uuid()	
applicant_user_id	UUID	NOT NULL, FK → users(id), UNIQUE	One institution per applicant account
name	TEXT	NOT NULL	Legal name of the institution

registration_number	TEXT	NOT NULL, UNIQUE	Issued by Rwanda Development Board or equivalent
phone	TEXT		Institution contact phone
address	TEXT		Registered physical address
created_at	TIMESTAMPTZ	NOT NULL DEFAULT now()	

6.2.3 user_workflow_roles

Links a STAFF user to the specific role they play within a given workflow. A staff member may hold different roles in different workflows. Assigned by Admin at account creation or updated later.

Column	Type	Constraints	Description
id	UUID	PK DEFAULT gen_random_uuid()	
user_id	UUID	NOT NULL, FK → users(id) ON DELETE CASCADE	The staff user
workflow_id	UUID	NOT NULL, FK → workflows(id) ON DELETE CASCADE	Which workflow this role applies to
role	TEXT	NOT NULL, CHECK IN ('INTAKE_OFFICER', 'REVIEWER','LEGAL_OFFICER', 'FINANCIAL_OFFICER','APPROVER')	Fine grained workflow role

assigned_by	UUID	NOT NULL, FK → users(id)	Admin who created this assignment
assigned_at	TIMESTAMPTZ	NOT NULL DEFAULT now()	

Constraint: UNIQUE(user_id, workflow_id, role), a user cannot hold the same role twice in the same workflow.

6.2.4 application types

Column	Type	Constraints	Description
id	UUID	PK	
name	TEXT	NOT NULL, UNIQUE	Human, readable type name e.g. Commercial Bank License
code	TEXT	NOT NULL, UNIQUE	Machine, readable identifier e.g. COMMERCIAL_BANK
description	TEXT		Shown to applicants when selecting a type
is_active	BOOLEAN	NOT NULL DEFAULT TRUE	Controls whether type is selectable for new applications
created_at	TIMESTAMPTZ	NOT NULL DEFAULT now()	

6.2.5 workflows

Column	Type	Constraints	Description
id	UUID	PK	
application_type_id	UUID	NOT NULL, FK → application_types(id)	Each type has exactly one workflow
name	TEXT	NOT NULL	
description	TEXT		

6.2.6 workflow_states

Column	Type	Constraints	Description
id	UUID	PK	
workflow_id	UUID	NOT NULL, FK -> workflows(id) ON DELETE CASCADE	
key	TEXT	NOT NULL	Machine key e.g. UNDER_LEGAL_REVIEW
label	TEXT	NOT NULL	Human label for UI display
is_initial	BOOLEAN	NOT NULL DEFAULT FALSE	Exactly one state per workflow must be initial

Column	Type	Constraints	Description
is_terminal	BOOLEAN	NOT NULL DEFAULT FALSE	Terminal states have no outgoing transitions
is_approved	BOOLEAN	NOT NULL DEFAULT FALSE	TRUE only on the positive terminal state
display_order	INTEGER	NOT NULL DEFAULT 0	UI ordering

Constraint: UNIQUE (workflow_id, key), state keys are unique within a workflow.

6.2.7 workflow_transitions

Column	Type	Constraints	Description
id	UUID	PK	
workflow_id	UUID	NOT NULL, FK -> workflows(id) ON DELETE CASCADE	
from_state_key	TEXT	NOT NULL	Must match a workflow_states.key in this workflow
to_state_key	TEXT	NOT NULL	Must match a workflow_states.key in this workflow

Column	Type	Constraints	Description
required_role	TEXT	NOT NULL, CHECK IN ('APPLICANT','INTAKE_OFFICER','REVIEWER', 'LEGAL_OFFICER','FINANCIAL_OFFICER','APPROVER')	Role that may trigger this transition
requires_decision	BOOLEAN	NOT NULL DEFAULT FALSE	If TRUE, a stage decision record is mandatory for this transition
label	TEXT		Button label in UI e.g. Forward to Legal Review

Constraint: UNIQUE (workflow_id, from_state_key, to_state_key).

6.2.8 document_requirements

Column	Type	Constraints	Description
id	UUID	PK	
application_type_id	UUID	NOT NULL, FK -> application_types(id) ON DELETE CASCADE	

Column	Type	Constraints	Description
key	TEXT	NOT NULL	Stable identifier e.g. audited_financials
label	TEXT	NOT NULL	Human label e.g. Audited Financial Statements
description	TEXT		Instructions shown to applicant
is_required	BOOLEAN	NOT NULL DEFAULT TRUE	Submission blocked if required and missing
allowed_mime_types	TEXT[]		Array of permitted MIME types e.g. {application/pdf}
max_size_bytes	INTEGER	DEFAULT 5242880	Default 5 MB; enforced server side
display_order	INTEGER	NOT NULL DEFAULT 0	

Constraint: UNIQUE (application_type_id, key).

6.2.9 applications

Column	Type	Constraints	Description
id	UUID	PK	
application_type_id	UUID	NOT NULL, FK - > application_types(id)	Immutable after creation
applicant_id	UUID	NOT NULL, FK -> users(id)	
workflow_id	UUID	NOT NULL, FK -> workflows(id)	Cached at creation, insulates from future workflow changes
current_state	TEXT	NOT NULL	Must match a workflow_states.key for this workflow
reviewed_by	UUID	FK -> users(id)	Set when first reviewer picks up application; enforces reviewer, cannot approve rule
version	INTEGER	NOT NULL DEFAULT 1	Optimistic locking counter, incremented on every state change

Column	Type	Constraints	Description
current_submission_cycle	INTEGER	NOT NULL DEFAULT 1	Increments each time applicant resubmits after PENDING_INFORMATION
submitted_at	TIMESTAMPTZ		Set when first moved to SUBMITTED
created_at	TIMESTAMPTZ	NOT NULL DEFAULT now()	
updated_at	TIMESTAMPTZ	NOT NULL DEFAULT now()	

6.2.10 application documents

Column	Type	Constraints	Description
id	UUID	PK	
application_id	UUID	NOT NULL, FK -> applications(id) ON DELETE RESTRICT	RESTRICT prevents accidental cascade delete
requirement_key	TEXT	NOT NULL	Matches document_requirements.key not a FK to allow historical stability
file_name	TEXT	NOT NULL	Original file name from upload

file_size	INTEGER	NOT NULL, CHECK (file_size <= 5242880)	DB level 5 MB guard in addition to server-side check
mime_type	TEXT	NOT NULL	
storage_path	TEXT	NOT NULL	Relative path on filesystem
uploaded_by	UUID	NOT NULL, FK -> users(id)	
submission_cycle	INTEGER	NOT NULL DEFAULT 1	Matches applications.current_submission_cycle at time of upload
superseded_by	UUID	FK -> application_documents(id)	Self-referential, points to the replacement document
uploaded_at	TIMESTAMPTZ	NOT NULL DEFAULT now()	

6.2.11 application stage reviews

One entry for each occasion the employee experiences a state while going through the application process. Atomic insert as part of the state transition transaction. Append,only; no UPDATE or DELETE operations permitted by application code against the table.

Column	Type	Constraints	Description
id	UUID	PK DEFAULT gen_random_uuid()	
application_id	UUID	NOT NULL, FK -> applications(id)	
workflow_state_key	TEXT	NOT NULL	The stage being completed e.g.

			UNDER_INITIAL_REVIEW
reviewed_by	UUID	NOT NULL, FK -> users(id)	Staff member completing this stage
decision_type	TEXT	NOT NULL, CHECK IN ('APPROVED_STAGE','REQUEST_INFO','ESCALATED')	Structured outcome of the stage review
decision_note	TEXT	NOT NULL, CHECK (length(trim(decision_note)) > 0)	Mandatory non-empty written rationale, visible to applicant
submission_cycle	INTEGER	NOT NULL DEFAULT 1	Links this review to the correct submission cycle
created_at	TIMESTAMPTZ	NOT NULL DEFAULT now()	Immutable, stage reviews are never edited

Constraint: UNIQUE (application_id, workflow_state_key, reviewed_by, submission_cycle) , only one record for each stage of each application assigned to each employee per submission cycle. When an application is recycled after REQUEST_INFO, the same employee can perform stage approval for this application in the new cycle.

Access Control: all records for stage review will be accessible to everyone involved, including the applicant. It is intentional. Transparency requirements require that an applicant must be able to view the rationale behind every decision applied to his/her application.

6.2.12 audit_logs

Column	Type	Constraints	Description
id	UUID	PK DEFAULT gen_random_uuid()	
application_id	UUID	FK -> applications(id)	NULL for system level events (login, user creation)
acting_user_id	UUID	NOT NULL, FK -> users(id)	
action	TEXT	NOT NULL	e.g. STATE_TRANSITION, STAGE_DECISION, DOCUMENT_UPLOAD, LOGIN, USER_CREATED
from_state	TEXT		Previous application state, NULL for non-transition events
to_state	TEXT		New application state, NULL for non-transition events
metadata	JSONB		Full application snapshot at time of action; IP address; session context
created_at	TIMESTAMPTZ	NOT NULL DEFAULT now()	Immutable record timestamp

Database level append only enforcement: REVOKE UPDATE, DELETE ON audit_logs FROM app_user; GRANT INSERT, SELECT ON audit_logs TO app_user

7. Relationships & Logical Data Model

7.1 Entity Relationships

Entity A	Cardinality	Entity B	Relationship Description
application_types	1 : 1	workflows	Each license type is governed by exactly one workflow
application_types	1 : many	document_requirements	Each type defines its own set of required documents
application_types	1 : many	applications	Many applications can be of the same type
workflows	1 : many	workflow_states	A workflow contains multiple states
workflows	1 : many	workflow_transitions	A workflow defines all valid transitions between its states
workflows	1 : many	applications	A workflow governs many applications (cached at creation)
workflows	1 : many	user_workflow_roles	Many staff members can be assigned roles within a workflow
users	1 : 1	institutions	Each applicant user is linked to exactly one institution

Entity A	Cardinality	Entity B	Relationship Description
users	1 : many	user_workflow_roles	A staff user can hold roles across multiple workflows
users	1 : many	applications	An applicant user may submit multiple applications
users	1 : many	application_documents	A user may upload many documents across many applications
users	1 : many	application_stage_reviews	A staff member records decisions across many application stages
users	1 : many	audit_logs	Every audit record is attributed to one acting user
applications	1 : many	application_documents	An application accumulates documents across submission cycles
applications	1 : many	application_stage_reviews	Each stage completion produces one review record per staff member per cycle
applications	1 : many	audit_logs	Every action on an application produces an audit record
application_documents	0 : 1	application_documents	Self,referential: superseded_by points to the replacement document

7.2 Logical Data Model - Key Invariants

The following invariants must be ensured through database constraints, application services' logic, and database privilege management:

• An application has a current_state value matching an existing workflow_states key for that particular workflow

• If an application has an is_terminal value of true, then the current_state value cannot be altered anymore

• The creation of one state change results in exactly one audit_log record – they cannot get out of sync

• The audit_log table contains no updates or deletions of audit log entries for all users of the database

• An application's reviewed_by value may not match the user_id of the acting user for a terminal approval/rejection transition

• An application_document's file_size cannot exceed 5,242,880 bytes (both on the server side and with CHECK constraints)

• Document entries are never removed from the system; superseding documents form a linked list

• Optimistic version counter prevents any two writers from transitioning the same application concurrently

• Stage decision must be recorded in all transitions in which workflow_transitions.requires_decision is set to TRUE – enforced by CHECK constraint (LENGTH(TRIM(decision_note)) > 0) in DB

• Once created, there is no way to change or delete stage decisions or related decision notes; stage decisions are append only

• Stage decisions with decision notes are accessible to all parties who have access, including the applicant

• User's workflow role is dynamically determined from user_workflow_roles in each request – never cached in the JWT

7.3 Indexes

Table	Index Columns	Type	Purpose
users	email	UNIQUE	Login lookup
institutions	applicant_user_id	UNIQUE	One institution per applicant
institutions	registration_number	UNIQUE	Duplicate institution guard
user_workflow_roles	user_id, workflow_id	B,Tree	Runtime workflow role resolution
applications	applicant_id	B,Tree	Applicant dashboard queries
applications	current_state, workflow_id	B,Tree	Staff filtered queue views
applications	application_type_id	B,Tree	Reporting queries
application_documents	application_id, requirement_key, submission_cycle	B,Tree	Document validation at submission

Table	Index Columns	Type	Purpose
application_stage_reviews	application_id, workflow_state_key, submission_cycle	B,Tree	Stage history and timeline queries
audit_logs	application_id, created_at	B,Tree	Audit history — chronological display
audit_logs	acting_user_id	B,Tree	User activity queries
workflow_transitions	workflow_id, from_state_key	B,Tree	State machine transition lookup

8. Data Specifications

8.1 API Response Envelope

All API responses use a consistent JSON envelope:

```
Success:   { "success": true,  "data": { ... } }
Error:     { "success": false, "error": { "code": "FORBIDDEN", "message": "..." } }
Paginated: { "success": true,  "data": [...], "pagination": { "page": 1, "limit": 20, "total": 87 } }
```

8.2 HTTP Status Code Specification

Status	Code	When Used
OK	200	Successful GET, PATCH, DELETE

Status	Code	When Used
Created	201	Successful POST creating a resource
No Content	204	Successful DELETE with no response body
Bad Request	400	Malformed JSON or missing required fields
Unauthorised	401	Missing or expired JWT token
Forbidden	403	Valid token but insufficient role for the action
Not Found	404	Requested resource does not exist
Conflict	409	Optimistic lock version mismatch on concurrent update
Unprocessable	422	Invalid state transition, empty decision note, or failed business rule
Server Error	500	Unhandled exception — no stack trace exposed; logged internally

8.3 Core Data Types & Validation Rules

Field	Type	Validation Rules
User email	TEXT	Valid email format; max 255 chars; lowercase,normalised before storage

Field	Type	Validation Rules
User password	TEXT	Min 8 chars; at least one uppercase, one number; never stored in plaintext
Institution reg. number	TEXT	Non,empty; unique across all institutions; stored as,is
Application type code	TEXT	Uppercase letters and underscores only e.g. COMMERCIAL_BANK
State key	TEXT	Uppercase letters and underscores only; must exist in workflow_states for this workflow
Stage decision_type	TEXT	Must be APPROVED_STAGE, REQUEST_INFO, or ESCALATED — validated in Joi before service layer
Stage decision_note	TEXT	Non,empty after trimming whitespace — enforced by Joi validation AND DB CHECK constraint
Document file size	INTEGER	Server,side multer limit 5,242,880 bytes; DB CHECK constraint also enforces this
Document MIME type	TEXT	Validated against allowed_mime_types array for the requirement; rejected if not in list
Submission cycle	INTEGER	Starts at 1; incremented only by service layer during PENDING_INFORMATION to SUBMITTED transition

Field	Type	Validation Rules
JWT access token expiry	Seconds	900 (15 minutes)
Refresh token expiry	Seconds	604800 (7 days)
Optimistic version	INTEGER	Starts at 1; monotonically increasing; never reset; never exposed to client
Audit metadata	JSONB	Serialised snapshot of the application row at time of action; immutable after insert

8.4 Application Timeline Response

The timeline endpoint combines state transitions and stage decisions into a single chronological feed visible to all authorised parties:

```
GET /api/applications/:id/timeline
{
 "timeline": [
   {
    "type":         "STATE_TRANSITION",
    "from_state":    "SUBMITTED",
    "to_state":      "UNDER_INITIAL_REVIEW",
    "actor":         { "id": "...", "full_name": "Alice Uwimana" },
    "created_at":    "2026,05,09T09:15:00Z"
   },
   {
    "type":         "STAGE_DECISION",
    "stage":         "UNDER_INITIAL_REVIEW",
    "decision_type": "APPROVED_STAGE",
```

```
    "decision_note": "All required documents received and verified.
                      Capital adequacy confirmed. Forwarding to legal review.",
    "reviewer":      { "id": "...", "full_name": "Alice Uwimana" },
    "cycle":         1,
    "created_at":    "2026,05,09T10:32:00Z"
  }
 ]
}
```


No role,based filtering is applied to the timeline, every entry is returned to every authorised party. The applicant sees the same timeline as the staff reviewer.

8.5 Seed Data Specification

The seed script creates the following users, workflow role assignments, and applications:

Email	Password	system_role	Workflow Role Assignments
applicant@bnr.rw	Test1234!	APPLICANT	None — applicants have no workflow roles
intake@bnr.rw	Test1234!	STAFF	INTAKE_OFFICER in Commercial Bank Workflow; INTAKE_OFFICER in Microfinance Workflow
legal@bnr.rw	Test1234!	STAFF	LEGAL_OFFICER in Commercial Bank Workflow; REVIEWER in Microfinance Workflow
financial@bnr.rw	Test1234!	STAFF	FINANCIAL_OFFICER in Commercial Bank Workflow

Email	Password	system_role	Workflow Role Assignments
approver@bnr.rw	Test1234!	STAFF	APPROVER in Commercial Bank Workflow; APPROVER in Microfinance Workflow
admin@bnr.rw	Test1234!	ADMIN	None — Admin operates at system level only

Application	Type	State	Seeded Stage Decisions
Test App 001	Commercial Bank License	PENDING_APPROVAL	Stage decisions from intake@bnr.rw, legal@bnr.rw, and financial@bnr.rw — each with written decision notes. Ready for approver@bnr.rw to action.
Test App 002	Microfinance Institution License	PENDING_INFORMATION	Stage decision from intake@bnr.rw with REQUEST_INFO decision and explanation. Ready for applicant@bnr.rw to

			resubmit documents.

9. Security Design Decisions

9.1 JWT vs Session,Based Authentication

JWT was selected over server,side sessions since this API is supposed to be stateless – JWTs have all claims within themselves and do not need server,side storage. The API tier is capable of scaling horizontally without any form of session,affinity or common session storage.

Limitation accepted: individual JWTs cannot be revoked prior to expiration using a denylist. The short,lived nature of 15 minutes access token solves this problem. In case where instant revocation is required, token denylist in Redis will be implemented in production.

9.2 Two Tier Role System

The system_role in the JWT (APPLICANT, STAFF, ADMIN) is deliberately crude. It dictates which routes a user has access to but does not define actions within applications. Workflow roles such as LEGAL_OFFICER and APPROVER are obtained by making requests to the service layer via the endpoint user_workflow_roles and are NEVER embedded in the token. That is to say that:

• The role of a Workflow for a particular user can be modified by an admin without requiring a logout from the user

• The JWT remains light and does not require refreshing in case of role changes in the organisation

• Role checking is performed in the service layer and not middleware since the latter has no knowledge of the workflow context of the application

9.3 Reviewer Cannot Be Approver

This rule is checked via 3 separate tiers:

• Data tier: applications.reviewed_by gets populated once the application is picked by the first reviewer

• Service tier: state machine verifies reviewed_by !== actingUser.id for all terminal actions and throws 403 otherwise

• User interface tier: the Approve/Reject button is greyed out for users whose id = reviewed_by, this is a fallback; the server,side validation is the authoritative one.

9.4 Append,Only Audit Log & Stage Reviews

Tamper resistance of both audit_logs and application_stage_reviews tables will be enforced as follows:

• Application level: no update or delete method is exposed in the repository layer on both tables

• Database level: REVOKE UPDATE, DELETE ON audit_logs FROM app_user – application code using the application role cannot update or delete audit log records

Privileges to be revoked for the stage reviews at the DB level cannot be the same as for the audit logs, since it is only an append only enforcement done via the application code. Reason: the stage review belongs to the application user and its risk level is much lower than that of the audit log.

9.5 Concurrent Access

Pessimistic locking is not preferred over optimistic locking (SELECT FOR UPDATE), as conflicts are not likely to occur at the same time within the same application, and optimistic locking does not hold locks in the database between reading and writing. The 409 Conflict is presented to the user with a message indicating that a refresh and retry is required.

10. Testing Strategy

Test Type	Scope	Tools	Key Scenarios
Unit, State Machine	stateMachine.js service	Jest	Valid transitions succeed; invalid transitions throw; terminal states block further transitions; reviewer,cannot,approve rule enforced
Unit, Stage Decision	stageDecision.js service	Jest	Empty decision_note rejected; whitespace,only note rejected; valid decision inserts correctly; missing decision blocks transition
Unit, RBAC	rbac middleware + workflow role resolution	Jest	Each system role reaches only its permitted routes; each workflow role triggers only its permitted transitions; cross,role attempts denied
Integration, Auth	POST /api/auth/login, /me, /logout	Supertest	Valid credentials return JWT; invalid credentials return 401; expired tokens return 401;

Test Type	Scope	Tools	Key Scenarios
Integration, Concurrency	POST /api/applications/:id/stage-decision	Jest + Supertest	Two simultaneous transition requests on same application, exactly one returns 200, one returns 409

11. Assumptions & Open Decisions

Ambiguity	Decision Made	Rationale
Are stage decision notes visible to the applicant?	Yes, all stage decisions and their written notes are universally visible	Regulatory transparency; applicants deserve clear written reasoning for every decision made on their application; eliminates complexity of a two,tier visibility system
Can a staff member edit a stage decision note after posting?	No, stage decisions are immutable after creation	Editing a regulatory decision record after the fact undermines trust in the review process; if clarification is needed, the reviewer completes a new stage in a subsequent cycle
What happens if BNR changes document requirements for a license	Changes apply to new applications only	Changing requirements mid,review is unfair to applicants; immutability at creation time is the safer regulatory position

Ambiguity	Decision Made	Rationale
type after applications are in flight?		
Can an applicant withdraw a submitted application?	No, once submitted it is in BNR's custody	Withdrawal rules are a regulatory policy question not specified; the conservative default is that submissions are irrevocable
Can an Admin see stage decisions and audit logs?	Yes, read only	Oversight requires full visibility; Admin cannot post stage decisions or modify audit records
Should APPROVER see applications before PENDING_APPROVAL?	No, Approver dashboard shows only applications requiring their action	Separation of concerns; Approver should form independent judgment without influence from earlier review notes
Can an applicant see another applicant's application?	No, queries always scoped to applicant_id = acting user	Regulatory confidentiality; one bank must not see another bank's pending license application
Notification emails on state changes?	Out of scope for v1	Requires email service and template management; documented as a future enhancement

12. Future Enhancements

- Notification emails and SMS messages regarding status changes and stages
- Redis based deny list for JSON Web Tokens that allows for quick token invalidation on user account hijacking or log out
- Approver to sign license document digitally with digital certificate.
- Export audit logs signed PDF and CSV format available for download for a complete audit record of a specific application
- Verification via email for the registration process of applicants
- Analytics dashboard processing time by license type, pipeline stage bottlenecks, rejections